1. Operators

*LAB EXERCISE 1: Simple Calculator
Write a C program that acts as a simple calculator. The program should take two numbers and an operator as input from the user and perform the respective operation (addition, subtraction, multiplication, division, or modulus) using operators.
* Challenge: Extend the program to handle invalid operator inputs.

```
#include <stdio.h>

int main() {
    double num1, num2;
    char operator;

    // Input
    printf("Enter first number: ");
    scanf("%lf", &num1);

    printf("Enter an operator (+, -, *, /, %%): ");
    scanf(" %c", &operator);

    printf("Enter second number: ");
    scanf("%lf", &num2);

    // Calculation
    switch(operator) {
        case '+':
            printf("Result = %.2lf\n", num1 + num2);
            break;

        case '-':
            printf("Result = %.2lf\n", num1 - num2);
            break;

        case '*':
            printf("Result = %.2lf\n", num1 * num2);
            break;

        case '/':
            if (num2 != 0)
                printf("Result = %.2lf\n", num1 / num2);
            else
                printf("Error: Division by zero is not allowed.\n");
            break;

        case '%':
            if ((int)num2 != 0)
                printf("Result = %d\n", (int)num1 % (int)num2);
            else
```

```
            printf("Error: Modulus by zero is not allowed.\n");
        break;

    // Challenge: Invalid operator handling
    default:
        printf("Error: Invalid operator entered.\n");
  }

  return 0;
}
```

LAB EXERCISE 2: Check Number Properties

*Write a C program that takes an integer from the user and checks the following using different operators:
o Whether the number is even or odd.
o Whether the number is positive, negative, or zero.
o Whether the number is a multiple of both 3 and

```
#include <stdio.h>

int main() {
  int num;

  // Take input from user
  printf("Enter an integer: ");
  scanf("%d", &num);

  // Check even or odd
  if (num % 2 == 0) {
    printf("The number is Even.\n");
  } else {
    printf("The number is Odd.\n");
  }

  // Check positive, negative, or zero
  if (num > 0) {
    printf("The number is Positive.\n");
  } else if (num < 0) {
    printf("The number is Negative.\n");
  } else {
    printf("The number is Zero.\n");
  }

  // Check multiple of both 3 and 5
  if (num % 3 == 0 && num % 5 == 0) {
    printf("The number is a multiple of both 3 and 5.\n");
  } else {
    printf("The number is NOT a multiple of both 3 and 5.\n");
  }
```

```
    return 0;
}
```

2. Control Statements

LAB EXERCISE 1: Grade Calculator

*Write a C program that takes the marks of a student as input and displays the corresponding grade based on the following conditions:
o Marks > 90: Grade A
o Marks > 75 and <= 90: Grade B
o Marks > 50 and <= 75: Grade C
o Marks <= 50: Grade D
*Use if-else or switch statements for the decision-making process.

```
#include <stdio.h>

int main() {
    int marks;

    // Input marks
    printf("Enter student marks: ");
    scanf("%d", &marks);

    // Grade calculation using if-else
    if (marks > 90) {
        printf("Grade A\n");
    }
    else if (marks > 75 && marks <= 90) {
        printf("Grade B\n");
    }
    else if (marks > 50 && marks <= 75) {
        printf("Grade C\n");
    }
    else {
        printf("Grade D\n");
    }

    return 0;
}
```

LAB EXERCISE 2: Number Comparison

*Write a C program that takes three numbers from the user and determines:
o The largest number.
o The smallest number.

*Challenge: Solve the problem using both if-else and switch-case statements.

```
#include <stdio.h>
```

```
int main() {
    int a, b, c;
    int largest, smallest;

    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    // Find largest
    if (a >= b && a >= c)
        largest = a;
    else if (b >= a && b >= c)
        largest = b;
    else
        largest = c;

    // Find smallest
    if (a <= b && a <= c)
        smallest = a;
    else if (b <= a && b <= c)
        smallest = b;
    else
        smallest = c;

    printf("Largest = %d\n", largest);
    printf("Smallest = %d\n", smallest);

    return 0;
}
```

3. Loops

LAB EXERCISE 1: Prime Number Check

*Write a C program that checks whether a given number is a prime number or not using a for loop.

*Challenge: Modify the program to print all prime numbers between 1 and a given number.

```
#include <stdio.h>

int main() {
    int num, i, isPrime = 1;

    printf("Enter a number: ");
    scanf("%d", &num);

    if (num <= 1) {
        isPrime = 0;
    } else {
        for (i = 2; i <= num / 2; i++) {
            if (num % i == 0) {
```

```
            isPrime = 0;
            break;
        }
    }
}

if (isPrime)
    printf("%d is a Prime number.\n", num);
else
    printf("%d is NOT a Prime number.\n", num);

return 0;
}
```

LAB EXERCISE 2: Multiplication Table

*Write a C program that takes an integer input from the user and prints its multiplication table using a for loop.
Challenge: Allow the user to input the range of the multiplication table (e.g., from 1 to N).

```
#include <stdio.h>

int main() {
    int num, i;

    // Take input from user
    printf("Enter an integer: ");
    scanf("%d", &num);

    // Print multiplication table from 1 to 10
    for(i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", num, i, num * i);
    }

    return 0;
}
```

LAB EXERCISE 3: Sum of Digits

*Write a C program that takes an integer from the user and calculates the sum of its digits using a while loop.
Challenge: Extend the program to reverse the digits of the number.

```
#include <stdio.h>

int main() {
    int num, originalNum, digit;
    int sum = 0, reverse = 0;

    printf("Enter an integer: ");
    scanf("%d", &num);
```

```
    originalNum = num;

    while (num != 0) {
        digit = num % 10;
        sum = sum + digit;
        reverse = reverse * 10 + digit;
        num = num / 10;
    }

    printf("Sum of digits = %d\n", sum);
    printf("Reversed number = %d\n", reverse);

    return 0;
}
```

4.Arrays

LAB EXERCISE 1: Maximum and Minimum in Array

*Write a C program that accepts 10 integers from the user and stores them in an array. The program should then find and print the maximum and minimum values in the array.
Challenge: Extend the program to sort the array in ascending order.

```
#include <stdio.h>

int main() {
    int arr[10];
    int i, max, min;


    printf("Enter 10 integers:\n");
    for(i = 0; i < 10; i++) {
        scanf("%d", &arr[i]);
    }

    max = min = arr[0];

    // Find max and min
    for(i = 1; i < 10; i++) {
        if(arr[i] > max) {
            max = arr[i];
        }
        if(arr[i] < min) {
            min = arr[i];
        }
    }


    printf("Maximum value = %d\n", max);
    printf("Minimum value = %d\n", min);
```

```
    return 0;
}
```

LAB EXERCISE 2: Matrix Addition

*Write a C program that accepts two 2x2 matrices from the user and adds them. Display the resultant matrix.
Challenge: Extend the program to work with 3x3 matrices and matrix multiplication.

```
#include <stdio.h>

int main() {
    int A[2][2], B[2][2], sum[2][2];
    int i, j;

    printf("Enter elements of first 2x2 matrix:\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            scanf("%d", &A[i][j]);
        }
    }

    printf("Enter elements of second 2x2 matrix
\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            scanf("%d", &B[i][j]);
        }
    }

    // Addition
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            sum[i][j] = A[i][j] + B[i][j];
        }
    }

    printf("Resultant Matrix (Sum):\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            printf("%d ", sum[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

LAB EXERCISE 3: Sum of Array Elements

*Write a C program that takes N numbers from the user and stores them in an array. The program should then calculate and display the sum of all array elements.
Challenge: Modify the program to also find the average of the numbers.

```
#include <stdio.h>

int main() {
   int n, i;
   int arr[100];
   int sum = 0;

   printf("Enter the number of elements: ");
   scanf("%d", &n);

   // Input elements
   printf("Enter %d numbers:\n", n);
   for(i = 0; i < n; i++) {
      scanf("%d", &arr[i]);
   }

   // Calculate sum
   for(i = 0; i < n; i++) {
      sum += arr[i];
   }

   printf("Sum of array elements = %d\n", sum);

   return 0;
}
```

5.Functions

LAB EXERCISE 1: Fibonacci Sequence

*Write a C program that generates the Fibonacci sequence up to N terms using a recursive function.
Challenge: Modify the program to calculate the Nth Fibonacci number using both iterative and recursive methods. Compare their efficiency.

```
#include <stdio.h>

// Recursive function to find Fibonacci number
int fibonacci(int n) {
   if (n <= 1)
      return n;
   return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
   int n, i;
```

```
    printf("Enter number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Sequence:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }

    return 0;
}
```

LAB EXERCISE 2: Factorial Calculation

*Write a C program that calculates the factorial of a given number using a function.
Challenge: Implement both an iterative and a recursive version of the factorial function and compare their performance for large numbers.

```
#include <stdio.h>
#include <time.h>

// Iterative factorial function
long long int factorial_iterative(int n) {
    long long int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}

// Recursive factorial function
long long int factorial_recursive(int n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial_recursive(n - 1);
}

int main() {
    int num;
    clock_t start, end;
    double time_taken;

    printf("Enter a number: ");
    scanf("%d", &num);

    // Iterative calculation
    start = clock();
    long long int result_iter = factorial_iterative(num);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
    printf("\n Iterative Factorial of %d = %lld", num, result_iter);
    printf("\n Time taken (Iterative): %f seconds\n", time_taken);

    // Recursive calculation
    start = clock();
    long long int result_rec = factorial_recursive(num);
    end = clock();
    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\n Recursive Factorial of %d = %lld", num, result_rec);
    printf("\n Time taken (Recursive): %f seconds\n", time_taken);

    return 0;
}
```

LAB EXERCISE 3: Palindrome Check

*Write a C program that takes a number as input and checks whether it is a palindrome using a function.
Challenge: Modify the program to check if a given string is a palindrome.

```
#include <stdio.h>

// Function to check palindrome number
int isPalindrome(int num) {
    int original = num;
    int reversed = 0, remainder;

    while (num != 0) {
        remainder = num % 10;
        reversed = reversed * 10 + remainder;
        num /= 10;
    }

    if (original == reversed)
        return 1;
    else
        return 0;
}

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (isPalindrome(number))
        printf("Palindrome number.\n");
    else
        printf("Not a palindrome number.\n");
```

```
    return 0;
}
```

6.Strings

LAB EXERCISE 1: String Reversal

*Write a C program that takes a string as input and reverses it using a function.
Challenge: Write the program without using built-in string handling functions.

```
#include <stdio.h>

// Function to reverse the string
void reverseString(char str[]) {
    int i = 0;
    int length = 0;
    char temp;

    // Find length manually
    while (str[length] != '\0') {
        length++;
    }

    // Reverse the string
    for (i = 0; i < length / 2; i++) {
        temp = str[i];
        str[i] = str[length - i - 1];
        str[length - i - 1] = temp;
    }
}

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Remove newline character manually
    int i = 0;
    while (str[i] != '\0') {
        if (str[i] == '\n') {
            str[i] = '\0';
            break;
        }
        i++;
    }

    reverseString(str);

    printf("Reversed string: %s\n", str);
```

```
    return 0;
}
```

LAB EXERCISE 2: Count Vowels and Consonants

*Write a C program that takes a string from the user and counts the number of vowels and consonants in the string.
Challenge: Extend the program to also count digits and special characters.

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[100];
    int vowels = 0, consonants = 0, digits = 0, special = 0, i = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    while (str[i] != '\0') {
        char ch = str[i];

        if (isalpha(ch)) {
            ch = tolower(ch);
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
                vowels++;
            } else {
                consonants++;
            }
        }
        else if (isdigit(ch)) {
            digits++;
        }
        else if (ch != ' ' && ch != '\n') {
            special++;
        }

        i++;
    }

    printf("\nVowels: %d", vowels);
    printf("\nConsonants: %d", consonants);
    printf("\nDigits: %d", digits);
    printf("\nSpecial Characters: %d\n", special);

    return 0;
}
```